

In Next.js's **Pages Router**, `getStaticProps` (Static Site Generation or SSG) generates HTML at **build time**, while `getServerSideProps` (Server-Side Rendering or SSR) generates HTML on **each request**. This difference determines the speed, data freshness, and deployment strategy for each page. [↗](#)

Feature ↗	<code>getStaticProps</code> (SSG)	<code>getServerSideProps</code> (SSR)
When it runs	At build time	On each user request
Data Freshness	Content is static until the next build (or revalidation).	Content is always up-to-date and real-time.
Performance	Extremely fast, as pages are cached by a CDN and served as static files.	Slower initial page load (higher Time to First Byte) because the server must compute the result for each request.
Use Cases	Content that changes infrequently (e.g., blog posts, marketing pages, documentation).	Data that is user-specific, highly dynamic, or requires access to request-time data (like cookies or headers).
SEO	Excellent for SEO due to fast page loads and pre-rendered HTML.	Also excellent for SEO, as content is fully pre-rendered on the server for crawlers.
Deployment	Requires a full rebuild and redeployment to update the page content (unless using Incremental Static Regeneration - ISR).	No rebuild needed to update content, as data is fetched on the fly.

Summary

- Use `getStaticProps` by default when possible to get the best performance and scalability. You can also add a `revalidate` property to enable [Incremental Static Regeneration \(ISR\)](#) to update pages periodically without a full rebuild.
- Use `getServerSideProps` only when you need data that is unique to a specific user's request and cannot be known ahead of time, such as personalized data or authorization information. [↗](#)

Note: In the newer Next.js **App Router** (the recommended way for new applications), these functions are replaced by React Server Components and a modern `fetch` API that handles both static and dynamic data fetching by default.